Greedy Goal

"At every step, I make a choice that looks best right now. Local best choices eventually give me global best answer"

Ex: I have ₹100. buy many items, prices [10, 20, 30, 40] which one choose first best result => ₹10 -> cheapest

Basic idea: At each step

Choose what looks best right now

"Greedy" -> Make the best choice available rn;

Local vs Global best

Amt = 30 Coins = [1, 5, 10, 20]

Task: Make the amt with fewest coins

1·20, 2·10 => 2 coins

we best choice at each step => Greedy

Local optimum	Global optimum	Greedy
Best choice right now	Best choice overall	Repeatedly choose local optimum

(*) Greedy does not always work

Ex: coins = [1, 3, 4] Amount: 6

Greedy -> 1·4, rem=2, 2·1, rem=1, 3·rem=1 => 4+1+1=6 (3 coins)

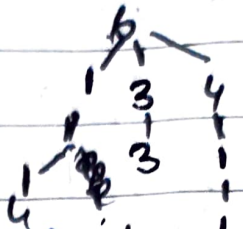
✓ Grt -> 1·3, rem=3, 2·3, rem=0, => 3+3=6 (2 coins, optimal)

"Greedy is Strategy, not guarantee"

Date :

Brute force coins = [1, 3, 4] amt = 6

↓ explore all possibilities



Bf 2

"What are all possibilities?"

Greedy → "what is the best choice right now?"

Greedy:	DP:
Choose now	consider different possibilities.
Don't reconsider	Store the best answer to smaller states

current situation

↓

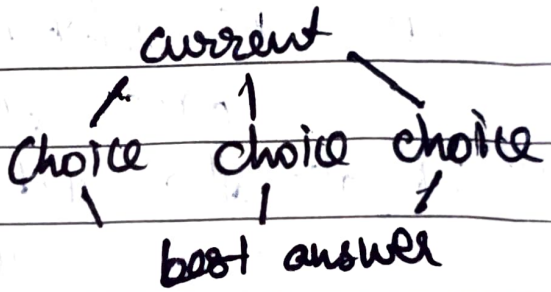
Best choice now

↓

next situation

↓

Best choice now



Greedy commits, DP compares

Assign Cookies

children = [1, 2, 3] cookies = [1, 1, 2, 3]

[cookie size >= child greed]

Greedy decision

→ "Give smallest cookie that can satisfy the current child," not bigger ones

↓

Save bigger for child who need

1-> Sort both arrays

2-> Move child if satisfies

3-> Always move the cookiepointer

Date : _____

LC 860 Lemonade Change → only 5, 10, 20 bills

Greedy: [5, 5, 5, 10, 20]

Sometimes the greedy choice is not simply largest or smallest

It is option that preserve future possibilities.

1. 5 → 5count → 1, 2. 5 → count-2, 3. 5count → 3

4. 10 → 5count 3-1, 10Count → 1,

5. 20 → 15 → 5 count 2-1, 10count → 1-1,

return true;

5 → increase 5count

10 → decrease 1 5count, increase 10count

Two possibilities 20 → 1. decrease 1 5count, 1 10count

2. if 3 5count, decrease 3 5count

else false

LC 121 Buy Sell Stock

min ↓ ↓ ↓ ↓ ↓ ↓

[7, 1, 5, 3, 6, 4]

maxProfit 0 0 4 4 5 5 ← output

→ Track ~~max~~ minimum price

1. 7 ⇒ min = 7, Profit = 0 | 2. 1 ⇒ min = 1, profit = 0

3. 5 ⇒ min = 1, profit = 4 | 4. 3 ⇒ min = 1, max profit = 4

5. 6 ⇒ min = 1, maxprofit = 5 | 6. 4 ⇒ min = 1, maxProfit = 5

return 5

Date : _____

LC1710 → Maximum Units on the truck

boxTypes = [no. of boxes, units per box]

truck = no. of boxes it can hold

return max no. of units truck can hold

Ex: [(1,3), (2,2) (3,1)] truck = 4

⇒ choose 1 box which one?

box with maximum units ⇒ 3 units

→ Sort box by unit descending

Already sorted

(1,3)

1. boxes = 0, units = 0 ⇒ boxes = 1, units = 3
2. (2,2) boxes = 3, units = 3+4 = 7
3. (3,1) → Only 1 space remaining

boxes = 4, units = 7+1 = 8

return units ⇒ 8 box full

Date :